

Implementation and Evaluation of a Lossy Spike Compression Framework for Spiking Neural Networks

Satvik GANESH s1302003

Supervised by Khanh N. DANG

Abstract

Spiking Neural Networks (SNNs) emulate the sparse and event-driven nature of biological neurons, offering significant potential for low-energy and efficient computation. However, the irregular and high-frequency spike activity in deep SNNs can introduce substantial communication and memory overhead. In this work, we present a spike compression method called Frequent Spike Pattern Compression (FSPC) that applies: (1) *correlated AER (Address Event Representation) merging* to reduce the bitwidth and (2) *approximation technique using pattern mining algorithms* to perform a lossy compression. Experimental results on benchmark SNN models and datasets demonstrate that our approach achieves up to 21.71% reduction in spike data size and an overall 0.97% drop in task accuracy when applied between the network's hidden layers. The source code for FSPC is publicly available at <https://github.com/klab-aizu/fspc>

1 Introduction

Spiking Neural Networks (SNNs) are the third generation of neural networks designed to mimic biological neurons [1]. Unlike conventional Artificial Neural Networks (ANNs), which transmit continuous values, SNNs use spiking neurons whose outputs are discrete firing events (spikes) generated when the membrane potential crosses a threshold [2]. Due to their event-driven, spike-based computation, SNNs promise greater energy efficiency than conventional ANNs [3].

SNNs process inputs as discrete events called spikes. While specialized sensors such as Dynamic Vision Sensors (DVS) can directly generate spike-based outputs suitable for SNNs [4, 5], most conventional data must first be converted into spike trains. This process, known as spike encoding [6], is typically categorized into rate coding and temporal coding.

Rate coding represents input intensity through spike frequency, with higher input values producing more spikes. It is simple, robust to noise, and widely used, but generates large spike volumes, increasing power

consumption and latency because sufficient spikes must be accumulated for accurate representation [7]. Temporal coding instead encodes information through the precise timing of spikes. As a result, it can represent the same information with fewer spikes, reducing power consumption and latency. However, it is more sensitive to input noise, and its reliance on spike timing makes it difficult to manage in distributed systems [7].

Given these trade-offs, this work focuses on rate coding because of its noise resilience while seeking to reduce the large data volume generated by spike trains. Excessive spiking activity increases power consumption and inference latency, limiting the scalability of SNNs in resource-constrained environments [8]. Previous work has explored module-wise compression [9] and spike inhibition [10]. However, the former primarily targets temporal encoding by exploiting spike timing, while the latter suppresses spikes during ANN-to-SNN conversion without addressing redundancy in rate-coded SNNs.

In neuromorphic systems, sparse spike activity makes Address Event Representation (AER) a common communication format [11]. For an A -axon system with only one active axon at a time, AER replaces the conventional A -bit spike vector with a $(1 + \log(A))$ -bit representation, reducing communication overhead between neuron groups.

To address these limitations, this thesis proposes *Frequent Spike Pattern Compression (FSPC)*, a compression framework for co-firing neuron groups. The framework was first presented at the 2025 IEEE MC-SoC conference [12]. FSPC uses pattern-mining algorithms to identify neuron subsets that frequently fire together or exhibit strong correlations. These subsets are then merged to reduce spike data volume. Furthermore, leveraging the fault tolerance of SNNs, which can tolerate minor deviations in spike timing, FSPC applies approximate subset AER compression to achieve additional compression. The key contributions are:

- A lossy spike compression framework that applies spatial approximations to reduce spike data vol-

ume.

- Comprehensive evaluation on benchmark SNN models and datasets to analyze compression ratio–accuracy trade-offs.

The organization of the paper is as follows. Section 2 introduces SNNs, rate coding, and the AER format. Section 3 describes the proposed compression method. Section 4 presents the experimental results, and Section 5 concludes the paper.

2 Background

This section will cover the background, such as the architecture of Spiking Neural Networks, the rate coding technique, and the Address Event Representation Format.

2.1 Architecture of SNN

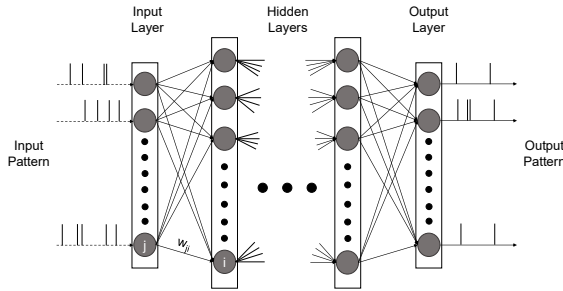


Figure 1: A typical SNN consists of an input layer, one or more hidden layers, and an output layer, where neuron j (presynaptic) connects to neuron i (postsynaptic) through a synaptic weight w_{ji} .

The fundamental structure of the SNN consists of a set of neurons connected to each other or, in other cases, recurrently to themselves. Fig. 1 displays a typical fully connected feed-forwarding model. The network consists of an input layer, one or more hidden layers, and an output layer. Each layer contains spiking neurons modeled using biologically inspired dynamics.

Several neuron models have been proposed to capture the spiking behavior with different levels of biological realism and computational cost. The Hodgkin-Huxley model shows a high accuracy in capturing the behavior of biological spikes, but it is computationally expensive to run [13]. The Izhikevich model offers a trade-off by reproducing a wide range of biological spiking patterns with relatively low computational

overhead [14]. Among these, the Leaky Integrate-and-Fire (LIF) [15] has become very popular as it balances biological plausibility with computational efficiency. A simplified equation of LIF is given below:

$$\tau \frac{dU(t)}{dt} = -U(t) + RI_{in}(t) \quad (1)$$

In this equation, $U(t)$ is the membrane potential, τ is the time constant, R is the membrane resistance, and $I_{in}(t)$ is the input current. When the membrane potential of a neuron exceeds its threshold V_{thres} , the neuron fires, and its membrane potential is reset to V_{reset} . Fig.2 illustrates the LIF membrane potential dynamics. From the figure, it can be seen that the application of input current $I_{in}(t)$ causes the membrane potential $U(t)$ to rise over time. Once it reaches the threshold V_{thres} , a spike occurs ($S_i = 1$).

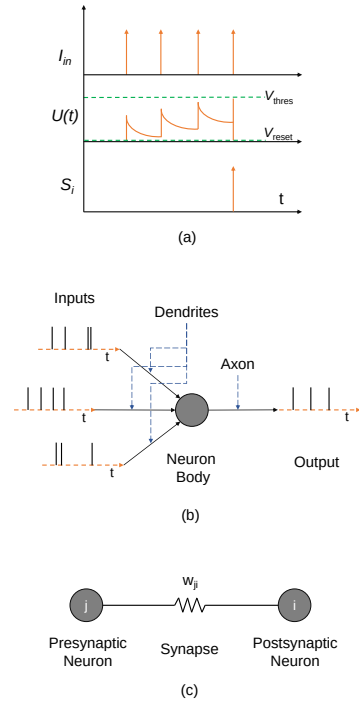


Figure 2: A typical neuron model: (a) Membrane potential $U(t)$ with the applied input current $I_{in}(t)$ until it reaches the threshold V_{thres} , after which it resets to V_{reset} , followed by a spike at $S_i = 1$. (b) Input spikes produced by I_{in} travel through the dendritic tree to the neuron body, and output spikes are emitted after adequate excitation. (c) The synaptic weight w_{ji} between neuron j and i .

The input current for input i is shown below:

$$I_i(t) = \sum_j w_{ji} S_j(t) \quad (2)$$

Here, w_{ji} is the synaptic weight of the presynaptic neuron j and $S_j(t)$ is its spike value at timestep t . These currents are injected into the input neurons over multiple timesteps. The firing and resetting of neuron i is shown as:

$$\text{If } U_i(t) \geq V_{\text{thres}}, \text{ then } \begin{cases} S_i(t) = 1, \\ U_i(t+1) = V_{\text{reset}} \end{cases} \quad (3)$$

Training of SNNs can be approached in several ways including [6] :

- Shadow training: A non-spiking ANN is initially trained and then converted into an SNN by interpreting activations as firing rates or spike timings.
- Backpropagation using spikes: The SNN is trained natively using error backpropagation, typically through time, similar to sequential models.
- Local learning rules: Weight updates are a function of signals that are spatially and temporally local to the weight, rather than from a global signal as in error backpropagation.

For classification tasks, the network's prediction is based on the output of the neuron that produces the most spikes over the inference window.

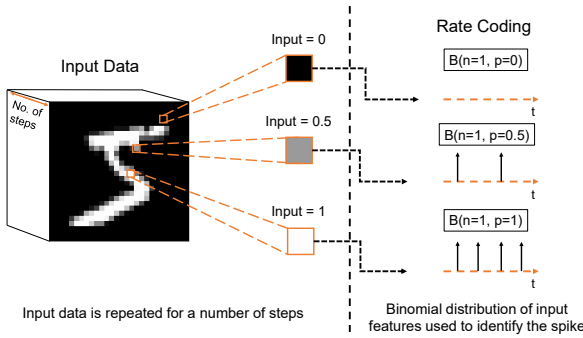


Figure 3: Rate coding for SNNs: An input MNIST image is encoded into spikes with varying intensities based on the Bernoulli distribution.

2.2 Coding Technique

Rate coding is a neural coding technique that converts input intensity into spiking frequency, or firing rate. Generally, an input sample is transformed into a rate-coded spike train as follows [6]: Let $\mathbf{M} \in \mathbb{R}^{x \times y}$

be an image sample, where x and y are the image dimensions. The goal is to convert $\mathbf{M}$ into a 3-D tensor $X \in \mathbb{R}^{x \times y \times t}$, where t is the number of timesteps. Each feature M_{ij} is encoded independently, with its normalized intensity (between 0 and 1) representing the probability of a spike occurring at a given timestep. This process is modeled as a Bernoulli trial, a special case of the binomial distribution, $X_{ijk} \sim B(n, p)$, where $n = 1$ and the spiking probability is $p = M_{ij}$. The probability of a spike is therefore:

$$P(X_{ijk} = 1) = M_{ij} = 1 - P(X_{ijk} = 0) \quad (4)$$

Sampling from this Bernoulli distribution for every feature at each timestep fills the 3-D tensor X with binary values (1s and 0s).

For an MNIST image, where $x = y = 28$, a pure white pixel ($M_{ij} = 1$) spikes with 100% probability, whereas a pure black pixel ($M_{ij} = 0$) never generates a spike. A grey pixel ($M_{ij} = 0.5$) has an equal probability of producing a spike or remaining silent. A visualization of rate coding is shown in Fig. 3. As the number of timesteps $t \rightarrow \infty$, the proportion of spikes is expected to converge to 0.5.

2.3 Address-Event Representation

Address-Event Representation (AER) is a neuromorphic communication protocol designed to encode the timing and location of sparse neural spike events [11, 16]. Instead of transmitting the entire binary spike matrix, AER only conveys the timing and location (spatiotemporal coordinates) of active spikes. Let $S_{t,n} \in \{0, 1\}^{T \times N}$ denote a binary spike matrix where T is the number of timesteps, N is the number of neurons, and $S_{t,n} = 1$ if neuron n spikes at time t , and 0 otherwise.

The AER represents only the indices where spikes occur:

$$A = \{(t, n) | S_{t,n} = 1\} \quad (5)$$

Each tuple $(t, n) \in A$ represents a spike event of a neuron n at timestep t .

3 Proposed Spike Compression Method

This section will describe the proposed spike compression method. First, an overview of the compression framework will be provided. Then, the pattern finding and compression phases are described. To understand more about how to find the patterns, some brief descriptions of the algorithms are shown.

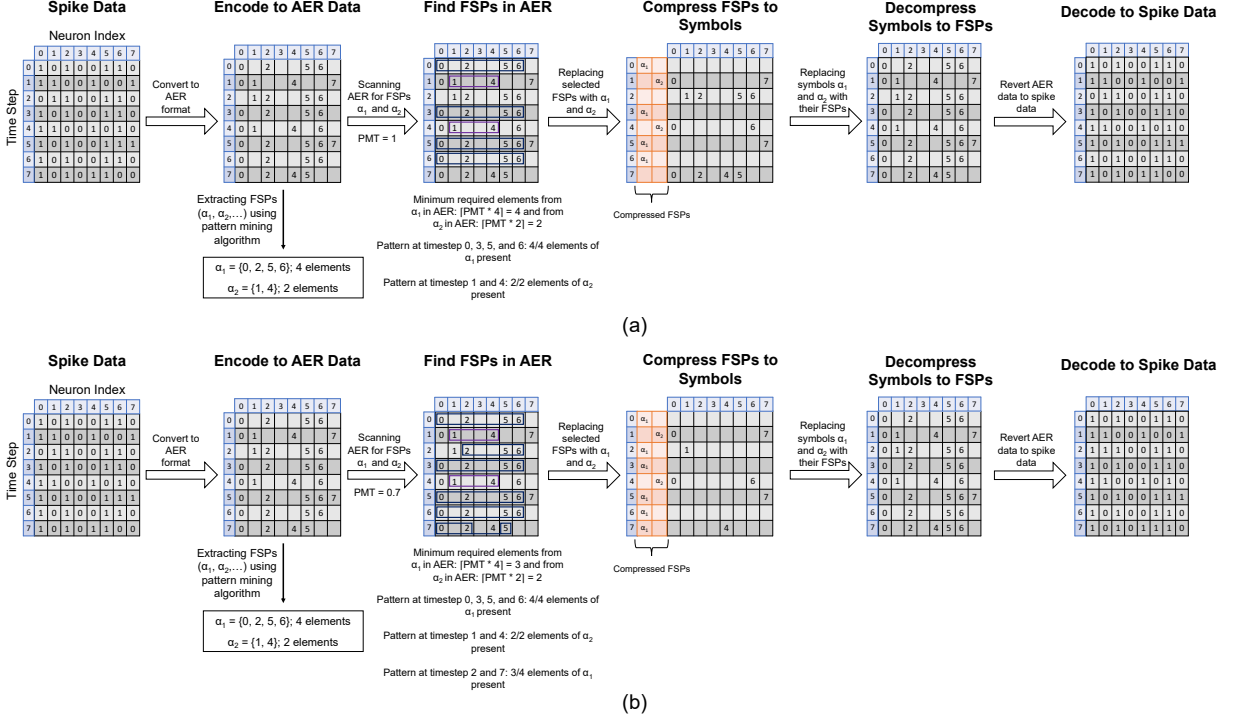


Figure 4: Compression of spikes using the proposed FSPC framework: (a) Using exact matching, i.e., PMT = 1. (b) Using a more relaxed matching, i.e., PMT = 0.7.

3.1 Compression Framework

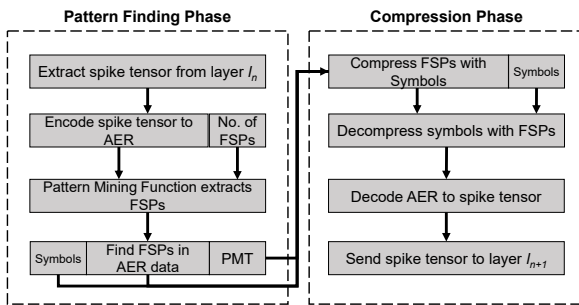


Figure 5: Frequent Spike Pattern Compression Framework.

The overview of the FSPC framework is shown in Fig. 5. This framework has two phases: the Pattern Finding Phase and the Compression Phase, and it currently works during the inference stage of the network.

3.2 Pattern Finding Phase

During the Pattern Finding Phase, spike tensor data from the previous layer is encoded into AER format. Mining algorithms such as fpmax from mlxtend [17], alongside FP-growth and maxFP-growth from PAMI [18], identify Frequent Spike Patterns (FSPs) of neurons firing simultaneously. Each FSP is assigned a unique symbol starting from a predefined base value. For example, in Fig. 4, the sets $\{0, 2, 5, 6\}$ and $\{1, 4\}$ are mapped to symbols α_1 and α_2 .

Once identified, the AER data is scanned for FSP occurrences based on a Pattern Match Threshold (PMT). This floating-point value (0 to 1) defines the minimum proportion of neurons required for a valid match. The number of neurons needed is calculated as:

$$N_{\text{required}} = \lceil \text{PMT} \cdot |\text{FSP Set}| \rceil \quad (6)$$

where $|\text{FSP Set}|$ is the number of neurons in the pattern and $\lceil \cdot \rceil$ denotes the ceiling function.

As shown in Fig. 4(a), when PMT = 1, α_1 requires all four neurons ($\lceil 1 \cdot 4 \rceil = 4$) and is detected at timesteps

0, 3, 5, and 6. Similarly, α_2 requires both neurons ($\lceil 1 \cdot 2 \rceil = 2$) at timesteps 1 and 4. If PMT is reduced to 0.7, the requirement for α_1 drops to three neurons ($\lceil 0.7 \cdot 4 \rceil = 3$), increasing its occurrences to timesteps 0, 2, 3, 5, 6, and 7. The requirement for α_2 remains at two neurons ($\lceil 0.7 \cdot 2 \rceil = 2$), leaving its detected timesteps unchanged.

3.3 Compression Phase

During the Compression Phase, the patterns detected for the presence of FSPs will be compressed by replacing the matched FSP with the corresponding symbol. Fig. 4 shows this for two PMT values:

- For PMT = 1, the patterns in timesteps 0, 3, 5, and 6 are replaced with α_1 and those at timesteps 1, 4 are replaced with α_2 (Fig. 4 (a))
- For PMT = 0.7, the patterns in timesteps 0, 2, 3, 5, 6, and 7 are replaced with α_1 and those at timesteps 1, 4 are replaced with α_2 (Fig. 4 (b)).

After compression, these symbols are decompressed back to their corresponding FSPs. For example, α_1 is replaced by {0, 2, 5, 6} and α_2 by {1, 4}. It should be noted that the compression can be lossy, depending on the PMT value, so the decompressed data may not be a replica of the original spike data. The decompressed AER is then decoded back to the spike format before being passed to the next layer.

3.4 Pattern Mining Algorithms

As explained in Section 3.2, three pattern mining algorithms are selected: fpmx by mlxtend (for low memory usage), FP-growth, and maxFP-growth (for fast speed) by PAMI. The implementation of FP-growth is based on the Frequent Pattern Growth algorithm [19], which compresses the dataset into a Frequent Pattern Tree (FP-Tree). This compact structure stores itemset frequency without generating candidate sets like Apriori [20]. The FP-tree is then traversed to find itemsets that appear frequently based on a user-defined minimum support threshold.

The maxFP-growth and fpmx implement variants of the FPMx algorithm [21, 22], which focuses on discovering Maximal Frequent Itemsets (MFIs). A frequent itemset is maximal if it has no superset that is frequent. Similar to FP-growth, the dataset is compressed into an FP-Tree. The tree is then recursively split into conditional FP-trees, with new itemsets checked against an MFI-tree to avoid redundant computation and efficiently identify the MFIs.

4 Evaluation Result

4.1 Experimental Methodology

In this experiment, a feedforward SNN based on the snnTorch framework was used, leveraging the backpropagation-through-time (BPTT) [23] training method. The MNIST dataset was used for evaluation. The tradeoff between compression and accuracy was compared for the increasing values of FSPs and PMT. The accuracy was defined by the number of correctly classified images from the test set. The compression percentage was calculated as:

$$\text{Compression} = 1 - \frac{\text{AER data length after FSPC}}{\text{AER data length before FSPC}} \cdot 100\% \quad (7)$$

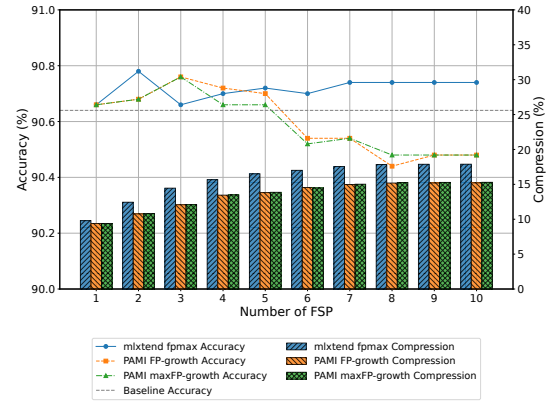


Figure 6: Evaluation of FSPC between layer HN-ON on a 784-100-10 neuron SNN with respect to FSP using the MNIST dataset.

Table 1: SNN Model Configurations for FSPC Framework

Model	Input Neurons (IN)	Hidden Neurons (HN)	Output Neurons (ON)
Three-layer	784	100	10
Four-layer	784	100 (HN1) 60 (HN2)	10

The compression framework has been tested on the configurations shown in Table 1. For the evaluation, 3,000 test images from the MNIST dataset were used, as the mining algorithms functioned reliably for this sample size. The minimum support value was set to 0.25. The experiments were carried out on an Ubuntu 22.04 machine with an Intel Core i7-13700K, NVIDIA RTX 4070, and 32 GB RAM.

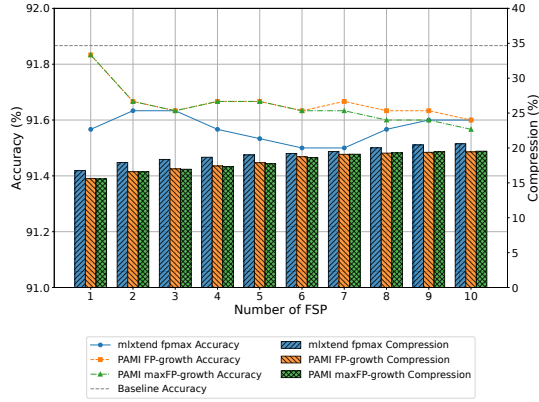


Figure 7: Evaluation of FSPC between layer HN1–HN2 in a 784-100-60-10 neuron SNN with respect to FSP using the MNIST dataset.

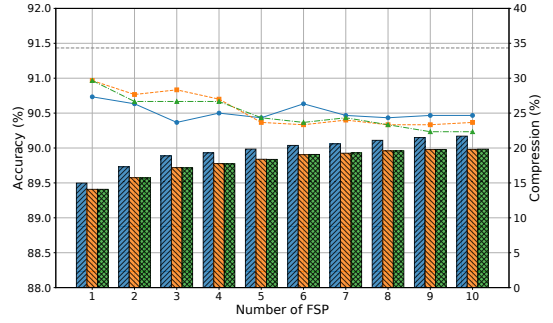
4.2 Evaluation Result based on FSP

This section analyzes how increasing the number of FSPs affects accuracy and compression for different mining algorithms, using a constant PMT of 0.65. Fig. 6 illustrates these effects on a 784-100-10 SNN with FSPC placed between the HN and ON layers. Accuracy decreases after 3 FSPs when using FP-growth and maxFP-growth, whereas mlxtend’s fpmax accuracy stagnates after 7 FSPs. For all algorithms, compression increases with the number of FSPs up to 8 before leveling off. Notably, accuracy with fpmax can exceed the baseline. This suggests a regularization effect where compressing frequent firing patterns removes noisy spikes and improves generalization.

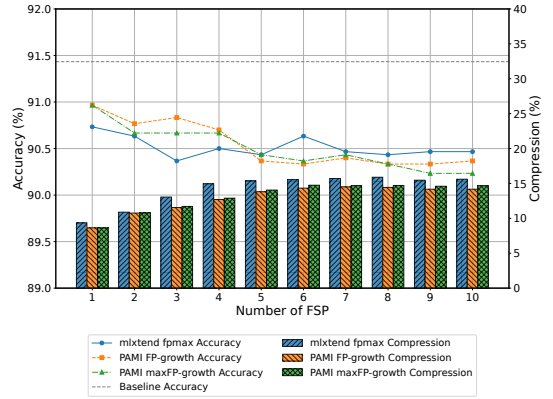
For the 784-100-60-10 SNN, placing FSPC between layers HN1 and HN2 resulted in an average accuracy drop of 0.26% from the baseline and 20.59% compression (Fig. 7). When FSPC was inserted between both the HN1–HN2 and HN2–ON layers (Fig. 8), overall accuracy decreased by up to 0.97%, while compression reached 21.71% for HN1–HN2 and 15.61% for HN2–ON layers.

4.3 Evaluation Result based on PMT

This section examines the effect of increasing the PMT value affects accuracy and compression for different pattern mining algorithms. A single FSP was used in all cases to isolate the impact of the parameter PMT. In all instances, accuracy increases with the PMT until it reaches the baseline. Fig. 9 shows this effect on the 784-100-10 SNN, where compression decreases from



(a) Between Layer HN1–HN2.



(b) Between Layer HN2–ON.

Figure 8: Evaluation of FSPC between layers HN1–HN2 and HN2–ON on a 784-100-60-10 neuron SNN using the MNIST dataset with respect to FSP.

12.62% to 5.25% as PMT increases.

When FSPC is inserted between both the HN1–HN2 and HN2–ON layers of the 784-100-60-10 SNN (Fig. 11), compression decreases from 18.58% to 6.77% in the HN1–HN2 layers and from 12.74% to 4.78% in the HN2–ON layers.

4.4 Discussion and Limitations

Experiments show mlxtend’s fpmax offers the best tradeoff between compression and accuracy. FSP values above 8 yield diminishing returns, while a PMT between 0.6 and 0.8 is optimal.

Key limitations of this study include unverified scalability on larger networks and datasets. Furthermore, we used data compression as a proxy for efficiency, and the mining algorithms’ computational overhead, a critical factor for hardware implementation, was not quantified. These are the vital directions for future work.

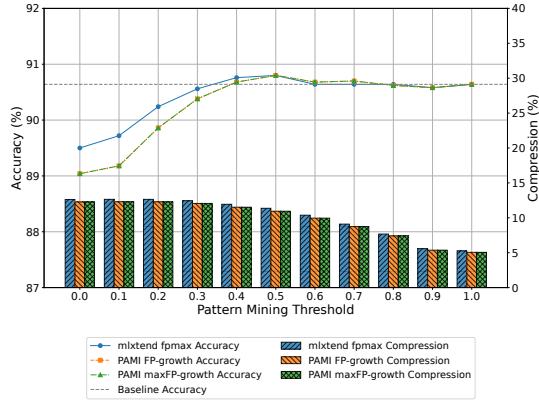


Figure 9: Evaluation of FSPC between layer HN-ON on a 784-100-10 neuron SNN with respect to PMT using the MNIST dataset.

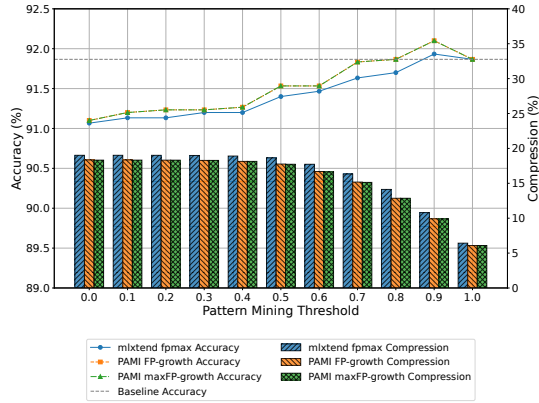
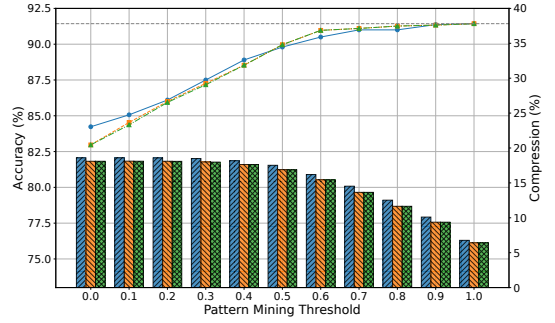


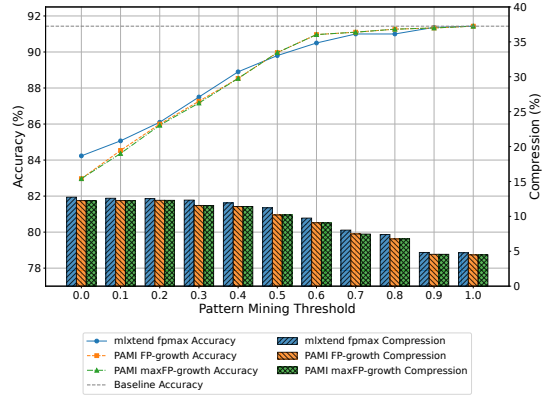
Figure 10: Evaluation of FSPC between layer HN and ON in a 784-100-60-10 neuron SNN with respect to PMT using the MNIST dataset.

5 Conclusion

This thesis presents FSPC, a lossy, software-based spike compression method for SNNs that reduces spike data volume by 12–15% with only a 1–3% accuracy loss in most cases. This work provides a foundation for efficient SNN simulation and points to future research in adaptive, hardware-aware compression. Future work includes implementing FSPC during the training phase for improved efficiency and developing custom mining algorithms to reduce runtime and scale to larger SNN models.



(a) Between HN1 and HN2



(b) Between Layer HN2 and ON.

Figure 11: Evaluation of FSPC between layers on a 784-100-60-10 neuron SNN using the MNIST dataset with respect to PMT.

Acknowledgements

I would like to thank my supervisor, Prof. Khanh N. Dang, for his guidance, ideas, and valuable feedback for this work. I am also grateful to Prof. Zhis-hang Wang and my colleague and friend, Hanyu Yuga, for their support and constructive feedback during the writing of this work.

References

- [1] A. B. Abdallah and K. N. Dang, *Neuromorphic computing principles and organization (1st edition)*, 1st ed. Springer, 2022.
- [2] W. Maass, “Networks of spiking neurons: The third generation of neural network models,” *Electron. Colloquium Comput. Complex.*, vol. TR96, 1996.

- [3] Z. Yan *et al.*, “Reconsidering the energy efficiency of spiking neural networks,” *ArXiv*, vol. abs/2409.08290, 2024.
- [4] P. Lichtsteiner *et al.*, “A 128×128 120 db 15 μ s latency asynchronous temporal contrast vision sensor,” *IEEE Journal of Solid-State Circuits*, vol. 43, no. 2, pp. 566–576, 2008.
- [5] R. Berner *et al.*, “A 240×180 10mw 12us latency sparse-output vision sensor for mobile applications,” in *2013 Symposium on VLSI Circuits*, 2013, pp. C186–C187.
- [6] J. K. Eshraghian *et al.*, “Training spiking neural networks using lessons from deep learning,” *Proceedings of the IEEE*, vol. 111, no. 9, pp. 1016–1054, 2023.
- [7] W. Guo *et al.*, “Neural coding in spiking neural networks: A comparative study for robust neuromorphic systems,” *Frontiers in Neuroscience*, vol. 15, p. 638474, 2021.
- [8] Y. Jiang *et al.*, “Stochastic spiking neural networks with first-to-spike coding,” *2024 International Conference on Neuromorphic Systems (ICONS)*, pp. 24–31, 2024.
- [9] L. Shi, A. Li, and B. Ward-Cherrier, “Spiking Brain Compression: Exploring One-Shot Post-Training Pruning and Quantization for Spiking Neural Networks,” in *OPT 2025: Optimization for Machine Learning*, 2025.
- [10] Z. Wang *et al.*, “Adaptive calibration: A unified conversion framework of spiking neural network,” *ArXiv*, vol. abs/2412.16219, 2023.
- [11] M. A. Mahowald, “VLSI analogs of neuronal visual processing: A synthesis of form and function,” PhD dissertation, California Institute of Technology, Pasadena, CA, USA, May 1992.
- [12] S. Ganesh, H. Yuga, Z. Wang, and K. N. Dang, “FSPC: A lossy spike compression through correlated-aer merging in spiking neural networks,” in *2025 IEEE 18th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoc)*. IEEE, 2025, pp. 386–393.
- [13] H. Al and H. Af, “A quantitative description of membrane current and its application to conduction and excitation in nerve,” *Bulletin of Mathematical Biology*, vol. 52, pp. 25–71, 1952.
- [14] E. M. Izhikevich, “Simple model of spiking neurons,” *IEEE transactions on neural networks*, vol. 14 6, pp. 1569–72, 2003.
- [15] L. Lapique, “Researches quantatives sur l’excitation electrique des nerfs traitee comme une polarization,” *Journal of Physiology, Pathology and Genetics*, vol. 9, pp. 620–635, 1907.
- [16] M. A. Sivilotti, *Wiring considerations in analog VLSI systems, with application to field-programmable networks*. California Institute of Technology, 1991.
- [17] S. Raschka, “Mlxtend: Providing machine learning and data science utilities and extensions to python’s scientific computing stack,” *The Journal of Open Source Software*, vol. 3, no. 24, Apr. 2018.
- [18] U. K. Rage *et al.*, “Pami: An open-source python library for pattern mining,” *Journal of Machine Learning Research*, vol. 25, no. 209, pp. 1–6, 2024.
- [19] J. Han *et al.*, “Mining frequent patterns without candidate generation,” in *ACM SIGMOD Conference*, 2000, pp. 1–12.
- [20] R. S. Agrawal and R. Srikant, “Fast algorithms for mining association rules,” in *Proceedings of the 20th International Conference on Very Large Data Bases, VLDB*, 1994, pp. 487–499.
- [21] G. Grahne and J. Zhu, “High performance mining of maximal frequent itemsets,” in *Proceedings of the 6th SIAM International Workshop on High Performance Data Mining*, 2003, p. 34.
- [22] Gösta Grahne and Jianfei Zhu, “Efficiently using prefix-trees in mining frequent itemsets,” in *Workshop on Frequent Itemset Mining Implementations*, 2003, p. 65.
- [23] P. Werbos, “Backpropagation through time: what it does and how to do it,” *Proceedings of the IEEE*, vol. 78, no. 10, pp. 1550–1560, 1990.